

Skill 9

Task 1

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
GNU nano 7.2 jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_JOBS 100
struct Job {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
// Add a new job
void add_job(pid_t pid, char command[]) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");
    job_count++;
}
// Update job status
void update_status() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
```

```

// List all active jobs
void list_jobs() {
    update_status();
    printf("\n-----\n");
    printf(" Job ID      PID      Status      Command\n");
    printf("-----\n");
    for (int i = 0; i < job_count; i++) {
        // Display only active jobs
        if (strcmp(jobs[i].status, "Running") == 0) {
            printf(" %-10d %-9d %-12s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].status,
                jobs[i].command);
        }
    }
    printf("-----\n");
}

// Display all jobs
void display_all_jobs() {
    update_status();
    printf("\n===== ALL JOBS =====\n");
    printf("%-8s %-10s %-15s %-20s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
    printf("-----\n");
}

```

```

}
int main() {
    int choice;
    char command[100];
    while (1) {
        printf("\n===== JOB MANAGER =====\n");
        printf("1. Start Job\n");
        printf("2. List Active Jobs\n");
        printf("3. Display All Jobs\n");
        printf("4. Update Status\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        // Start a new job
        if (choice == 1) {
            printf("Enter command: ");
            fgets(command, sizeof(command), stdin);
            command[strcspn(command, "\n")] = '\0';
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork");
            }
            else if (pid == 0) {
                execl("/bin/sh",
                    "sh",
                    "-c",

```

```
GNU nano 7.2                                jobs.c *
}
else if (pid == 0) {
    execl("/bin/sh",
          "sh",
          "-c",
          command,
          NULL);
    perror("exec");
    exit(1);
}
else {
    add_job(pid, command);
    printf("\nJob started successfully!\n");
    printf("Job ID: %d\n", job_count);
    printf("PID: %d\n", pid);
}
}
// List active jobs
else if (choice == 2) {
    list_jobs();
}
// Display all jobs
else if (choice == 3) {
    display_all_jobs();
}
// Update job status
else if (choice == 4) {
    update_status();
    printf("\nJob status updated successfully.\n");
}
```

Output:

```
root@Tejasri:~# nano jobs.c
root@Tejasri:~# gcc jobs.c -o jobs
root@Tejasri:~# ./jobs

===== JOB MANAGER =====
1. Start Job
2. List Active Jobs
3. Display All Jobs
4. Update Status
5. Exit
Enter choice: 4

Job status updated successfully.

===== JOB MANAGER =====
1. Start Job
2. List Active Jobs
3. Display All Jobs
4. Update Status
5. Exit
Enter choice: 5
Exiting...
root@Tejasri:~#
```

Task 2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
int main() {
    char command[100];
    printf("Enter foreground command: ");
    fgets(command, sizeof(command), stdin);
    // Remove newline
    command[strcspn(command, "\n")] = '\0';
    // Create child process
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        // Create a new process group
        setpgid(0, 0);
        // Resume the process if it was stopped
        kill(getpid(), SIGCONT);
        // Execute command
        execl("/bin/sh", "sh", "-c", command, NULL);
        perror("exec");
        exit(1);
    }
    else {
        // Parent creates child's process group

```

```

        * Give terminal control to child
        */
        tcsetpgrp(STDIN_FILENO, pid);
        printf("Terminal control transferred to job.\n");
        int status;
        /*
        * Wait for foreground job
        */
        waitpid(pid, &status, WUNTRACED);
        /*
        * Take terminal control back
        */
        tcsetpgrp(STDIN_FILENO, getpgrp());
        /*
        * Check job state
        */
        if (WIFEXITED(status)) {
            printf("\nJob completed.\n");
            printf("Exit status: %d\n",
                WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status)) {
            printf("\nJob terminated by signal.\n");
        }
        else if (WIFSTOPPED(status)) {
            printf("\nJob stopped.\n");
            // Resume the stopped process
            kill(pid, SIGCONT);
            printf("Job resumed.\n");
        }
    }
}

```

Output:

```
root@Tejasri:~# nano ground.c
root@Tejasri:~# gcc ground.c -o ground
root@Tejasri:~# ./ground
Enter foreground command: sleep 20

Job started.
Job PID: 6712
Terminal control transferred to job.

Job completed.
Exit status: 0
root@Tejasri:~# |
```

Task 2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_JOBS 100
struct Job {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
/* Add a new job */
void add_job(pid_t pid, char command[]) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");
    job_count++;
}
/* Update status of all jobs */
void update_status() {
    for (int i = 0; i < job_count; i++) {
```

```

    */
    int status;
    pid_t result = waitpid(
        jobs[i].pid,
        &status,
        WNOHANG
    );
    if (result > 0) {
        if (WIFEXITED(status)) {
            strcpy(jobs[i].status, "Completed");
        }
        else if (WIFSIGNALED(status)) {
            strcpy(jobs[i].status, "Terminated");
        }
    }
}

/* List active jobs */
void list_active_jobs() {
    update_status();
    printf("\n");
    printf("=====\n");
    printf("                ACTIVE JOBS\n");
    printf("=====\n");
    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
}

```

```

printf("-----\n");
int found = 0;
for (int i = 0; i < job_count; i++) {
    if (strcmp(jobs[i].status, "Running") == 0) {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
        found = 1;
    }
}
if (!found) {
    printf("No active jobs.\n");
}
printf("=====\n");
}

/* Display all jobs */
void display_all_jobs() {
    update_status();
    printf("\n");
    printf("=====\n");
    printf("                ALL JOBS\n");
    printf("=====\n");
    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
}

```

```

}
/* Main program */
int main() {
    int choice;
    char command[100];
    while (1) {
        printf("\n===== JOB MANAGER =====\n");
        printf("1. Start Background Job\n");
        printf("2. List Active Jobs\n");
        printf("3. Display All Jobs\n");
        printf("4. Update Job Status\n");
        printf("5. Exit\n");
        printf("=====\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        /* Start background job */
        if (choice == 1) {
            printf("Enter command: ");
            fgets(command,
                sizeof(command),
                stdin);
            command[strcspn(command, "\n")] = '\0';
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork");
            }
            else if (pid == 0) {
                /*

```

```

                * Child executes command.
                */
                execl("/bin/sh",
                    "sh",
                    "-c",
                    command,
                    NULL);
                perror("exec");
                exit(1);
            }
            else {
                /*
                 * Parent does NOT wait.
                 * Therefore this is a background job.
                 */
                add_job(pid, command);
                printf("\nJob started successfully.\n");
                printf("Job ID : %d\n", job_count);
                printf("PID    : %d\n", pid);
            }
        }
        /* List active jobs */
        else if (choice == 2) {
            list_active_jobs();
        }
        /* Display all jobs */
        else if (choice == 3) {
            display_all_jobs();
        }
    }
}

```

```

    }
    /* Display all jobs */
    else if (choice == 3) {
        display_all_jobs();
    }
    /* Update status */
    else if (choice == 4) {
        update_status();
        printf("\nJob status updated successfully.\n");
    }
    /* Exit */
    else if (choice == 5) {
        printf("Exiting...\n");
        break;
    }
    else {
        printf("Invalid choice.\n");
    }
}
return 0;
}

```

Output:

```

root@Tejasri:~# gcc foreground.c -o foreground
root@Tejasri:~# ./foreground

===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
=====
Enter choice: 2

=====
                ACTIVE JOBS
=====
Job ID   PID      Status      Command
-----
No active jobs.
=====

===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
=====
Enter choice: 3

=====
                ALL JOBS
=====
Job ID   PID      Status      Command

```

```
=====
===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
=====
Enter choice: 3

=====
ALL JOBS
=====
Job ID   PID      Status      Command
-----
=====

===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
=====
Enter choice: 5
Exiting...
root@Tejasri:~# |
```

Task 3

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
/* Signal handler */
void signal_handler(int sig) {
    if (sig == SIGINT) {
        printf("\nSIGINT received (Ctrl+C)\n");
    }
    else if (sig == SIGTERM) {
        printf("\nSIGTERM received\n");
    }
    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received\n");
    }
    fflush(stdout);
}
int main() {
    pid_t pid;
    sigset_t mask;
    int status;
    printf("Parent Process\n");
    printf("PID : %d\n", getpid());
    printf("PGID : %d\n", getpgrp());
    /* Register signal handlers */
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);
```

```

/* Register signal handlers */
signal(SIGINT, signal_handler);
signal(SIGTERM, signal_handler);
signal(SIGUSR1, signal_handler);
/* Create signal mask */
sigemptyset(&mask);
/* Add SIGUSR1 to mask */
sigaddset(&mask, SIGUSR1);
/* Block SIGUSR1 */
sigprocmask(SIG_BLOCK, &mask, NULL);
printf("\nSIGUSR1 is blocked.\n");
/* Create child process */
pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}
if (pid == 0) {
    printf("\nChild Process\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("PGID : %d\n", getpgrp());
    printf("Child waiting for signals...\n");
    /* Wait for some time */
    sleep(10);
    printf("Child finished.\n");
    exit(0);
}
else {}

```

```

}
else {
    printf("\nChild created.\n");
    printf("Child PID: %d\n", pid);
    /* Send SIGUSR1 to child */
    printf("\nSending SIGUSR1 to child...\n");
    kill(pid, SIGUSR1);
    printf("SIGUSR1 sent.\n");
    /* Send SIGTERM after a delay */
    sleep(2);
    printf("\nSending SIGTERM to child...\n");
    kill(pid, SIGTERM);
    /* Wait for child */
    waitpid(pid, &status, 0);
    printf("\nChild process completed.\n");
}
return 0;
}

```

Output:

```
root@Tejasri:~# nano signals.c
root@Tejasri:~# nano Signals.c
root@Tejasri:~# gcc Signals.c -o Signals
root@Tejasri:~# ./Signals
Parent Process
PID : 6980
PGID : 6980

SIUSR1 is blocked.

Child created.
Child PID: 6981

Sending SIGUSR1 to child...
SIGUSR1 sent.

Child Process
PID : 6981
PPID : 6980
PGID : 6980
Child waiting for signals...

Sending SIGTERM to child...

SIGTERM received
Child finished.

Child process completed.
root@Tejasri:~# |
```

Task 4

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#define MAX_JOBS 10
struct Job {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
/* Add job to job table */
void add_job(pid_t pid, char command[]) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Stopped");
    job_count++;
}
/* Display job table */
void display_jobs() {
    printf("\n=====\\n");
    printf("          JOB TABLE\\n");
    printf("=====\\n");
    printf("%-8s %-10s %-15s %s\\n",

```

```

/* Resume selected job */
void resume_job(int job_id) {
    if (job_id < 1 || job_id > job_count) {
        printf("Invalid Job ID.\\n");
        return;
    }
    int index = job_id - 1;
    printf("\\nSending SIGCONT to PID %d...\\n",
        jobs[index].pid);
    if (kill(jobs[index].pid, SIGCONT) == -1) {
        perror("kill");
        return;
    }
    /* Update status */
    strcpy(jobs[index].status, "Running");
    printf("Job %d resumed successfully.\\n",
        jobs[index].job_id);
}
/* Check whether job is completed */
void update_status() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
        if (result > 0) {
            if (WIFEXITED(status)) {

```

```

}
int main() {
    int choice;
    int job_id;
    char command[100];
    while (1) {
        printf("\n===== JOB MANAGER =====\n");
        printf("1. Start Job\n");
        printf("2. Display Jobs\n");
        printf("3. Resume Stopped Job\n");
        printf("4. Update Job Status\n");
        printf("5. Exit\n");
        printf("===== \n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        /* Start job */
        if (choice == 1) {
            printf("Enter command: ");
            fgets(command,
                sizeof(command),
                stdin);
            command[strcspn(command, "\n")] = '\0';
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork");
                continue;
            }
            if (pid != 0) {

```

```

}
        /* Display jobs */
        else if (choice == 2) {
            update_status();
            display_jobs();
        }
        /* Resume job */
        else if (choice == 3) {
            printf("Enter Job ID to resume: ");
            scanf("%d", &job_id);
            resume_job(job_id);
        }
        /* Update status */
        else if (choice == 4) {
            update_status();
            printf("Job status updated.\n");
            display_jobs();
        }
        /* Exit */
        else if (choice == 5) {
            printf("Exiting...\n");
            break;
        }
        else {
            printf("Invalid choice.\n");
        }
    }
    return 0;
}

```

Output:

```
root@Tejasri:~# nano resume.c
root@Tejasri:~# gcc resume.c -o resume
root@Tejasri:~# ./resume
```

```
===== JOB MANAGER =====
```

1. Start Job
2. Display Jobs
3. Resume Stopped Job
4. Update Job Status
5. Exit

```
=====
```

```
Enter choice: 1
```

```
Enter command: sleep 10
```

```
Job created.
```

```
Job ID: 1
```

```
PID: 7048
```

```
===== JOB MANAGER =====
```

1. Start Job
2. Display Jobs
3. Resume Stopped Job
4. Update Job Status
5. Exit